Memory Management

Scoping and Memory

- Whenever we declare a new variable (int x), memory is allocated
- When can this memory be freed up (so it can be used to store other variables)?
  - When the variable goes out of scope

Scoping and Memory

- When a variable goes out of scope, that memory is no longer guaranteed to store the variable's value

```
int main() {
  if (true) {
    int x = 5;
  }
  // x now out of scope, memory it used to occupy can be reused
}
```

Scoping and Memory

- When a variable goes out of scope, that memory is no longer guaranteed to store the variable's value

```
int main() {
  int *p;
  if (true) {
    int x = 5;
    p = &x;
  }
  cout << *p << endl; // ???
}
```

Scoping and Memory

- When a variable goes out of scope, that memory is no longer guaranteed to store the variable's value

```
int main() {
  int *p;
  if (true) {
    int x = 5;
    p = &x;
  }
  cout << *p << endl; // ???
}
```



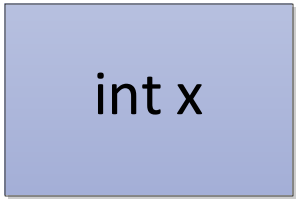
int *p

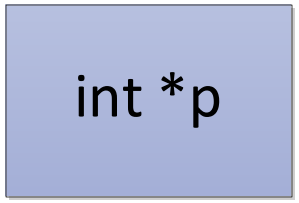
Scoping and Memory

- When a variable goes out of scope, that memory is no longer guaranteed to store the variable's value

```
int main() {
  int *p;
  if (true) {
    int x = 5;
    p = &x;
  }
  cout << *p << endl; // ???
}
```





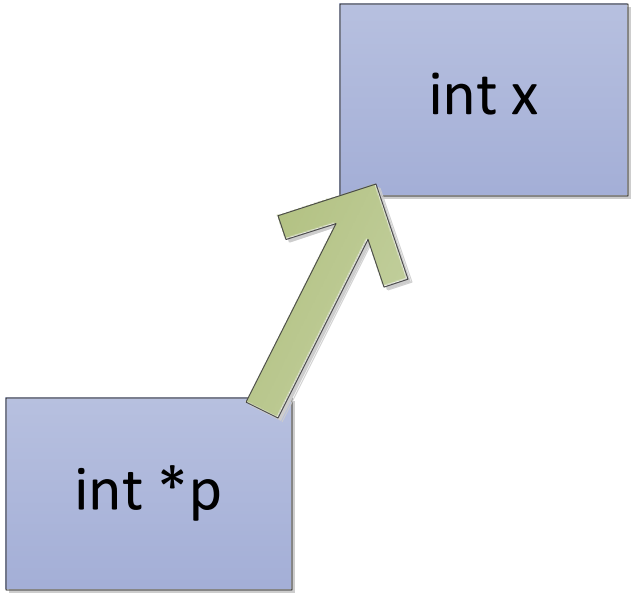


Scoping and Memory

- When a variable goes out of scope, that memory is no longer guaranteed to store the variable's value

```
int main() {
  int *p;
  if (true) {
    int x = 5;
    p = &x;
  }
  cout << *p << endl; // ???
}
```

here

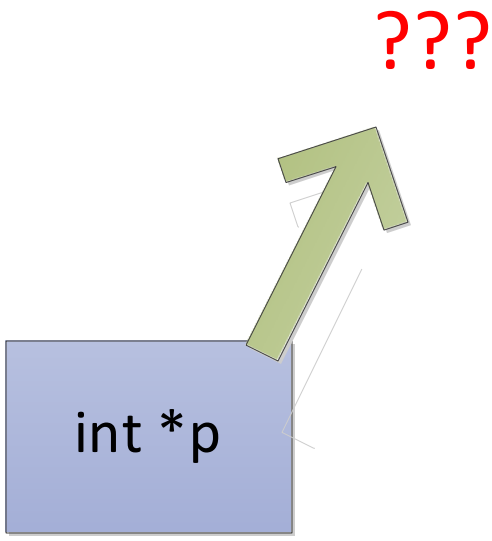


Scoping and Memory

- When a variable goes out of scope, that memory is no longer guaranteed to store the variable's value
  - Here, p has become a **dangling pointer** (points to memory whose contents are undefined)

```
int main() {
  int *p;
  if (true) {
    int x = 5;
    p = &x;
  }
  cout << *p << endl; // ???
}
```

here



A Problematic Task

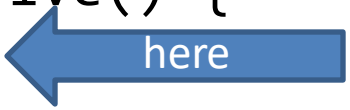
- Implement a function which returns a pointer to some memory containing the integer 5
- Incorrect implementation:

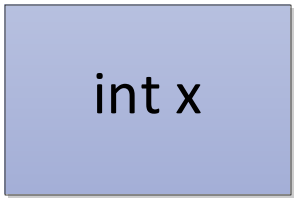
```
int* getPtrToFive() {
  int x = 5;
  return &x;
}
```

- Implement a function which returns a pointer to some memory containing the integer 5
- Incorrect implementation:
  - x is declared in the function scope

```
int* getPtrToFive() {
  int x = 5;
  return &x;
}

int main() {
  int *p = getPtrToFive();
  cout << *p << endl; // ???
}
```



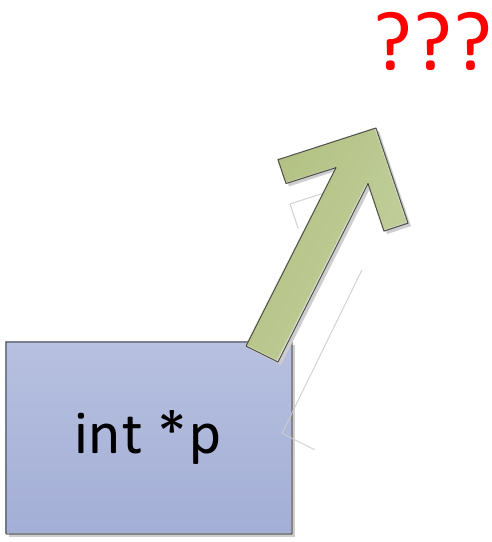


- Implement a function which returns a pointer to some memory containing the integer 5
- Incorrect implementation:
  - x is declared in the function scope
  - As getPtrToFive() returns, x goes out of scope. So a dangling pointer is returned

```
int* getPtrToFive() {
  int x = 5;
  return &x;
}

int main() {
  int *p = getPtrToFive();
  cout << *p << endl; // ???
}
```

here



The **new** operator

- Another way to allocate memory, where the memory will remain allocated until you manually de-allocate it
- Returns a pointer to the newly allocated memory

```
int *x = new int;
```

The **new** operator

- Another way to allocate memory, where the memory will remain allocated until you manually de-allocate it
- Returns a pointer to the newly allocated memory

```
int *x = new int;
```

Type parameter needed to determine how much memory to allocate

The **new** operator and Brackets!

-initializing a new int

```
int *x = new int(5);
```

```
int *x = new int{5};
```

See later what [] does

```
int *x = new int[5];
```

The **new** operator

- Another way to allocate memory, where the memory will remain allocated until you manually de-allocate it
- Returns a pointer to the newly allocated memory
- Terminology note:
  - If using **int x;** the allocation occurs on a region of memory called **the stack**
  - If using **new int;** the allocation occurs on a region of memory called **the heap**

The **delete** operator

- De-allocates memory that was previously allocated using **new**
- Takes a pointer to the memory location

```
int *x = new int;
// use memory allocated by new
delete x;
```

- Implement a function which returns a pointer to some memory containing the integer 5
  - Allocate memory using **new** to ensure it remains allocated

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}
```

- Implement a function which returns a pointer to some memory containing the integer 5
  - Allocate memory using **new** to ensure it remains allocated.
  - When done, de-allocate the memory using **delete**

```
int *getPtrToNewInt() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int*p= getPtrToNewInt();
  cout << *p << endl; // 5
  delete p;
}
```

Delete Memory When Done Using It

- If you don't use de-allocate memory using **delete**, your application will waste memory

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int  *p;
  for  (int i =   0; i  <  3;i++)
    p = getPtrToFive();
    cout << *p << endl;
  }
}
```



- If you don’t use de-allocate memory using **delete**, your application will waste memory

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
  }
}
```

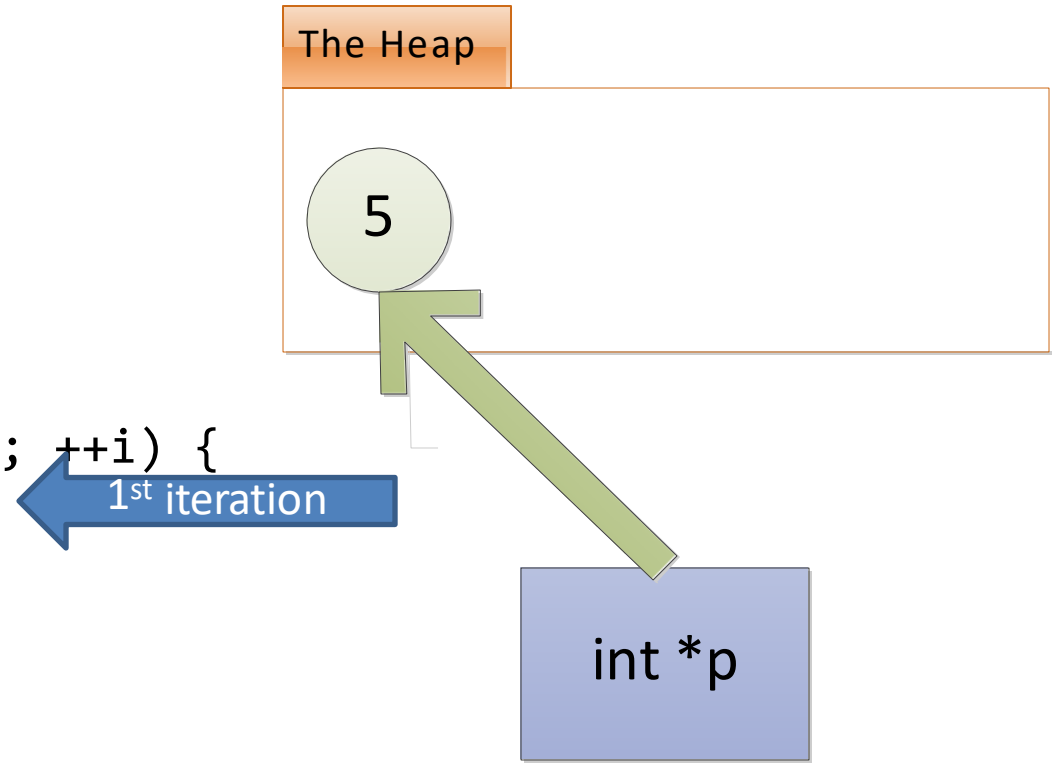


int *p

- If you don't use de-allocate memory using **delete**, your application will waste memory

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

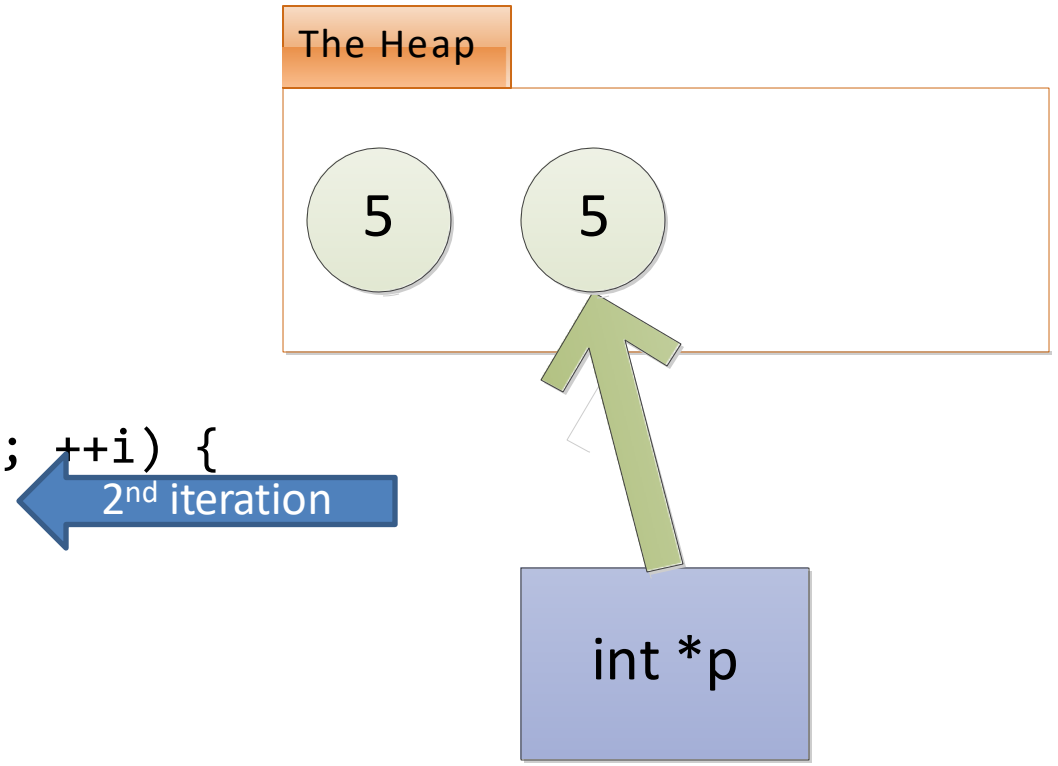
int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
  }
}
```



- If you don't use de-allocate memory using **delete**, your application will waste memory

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

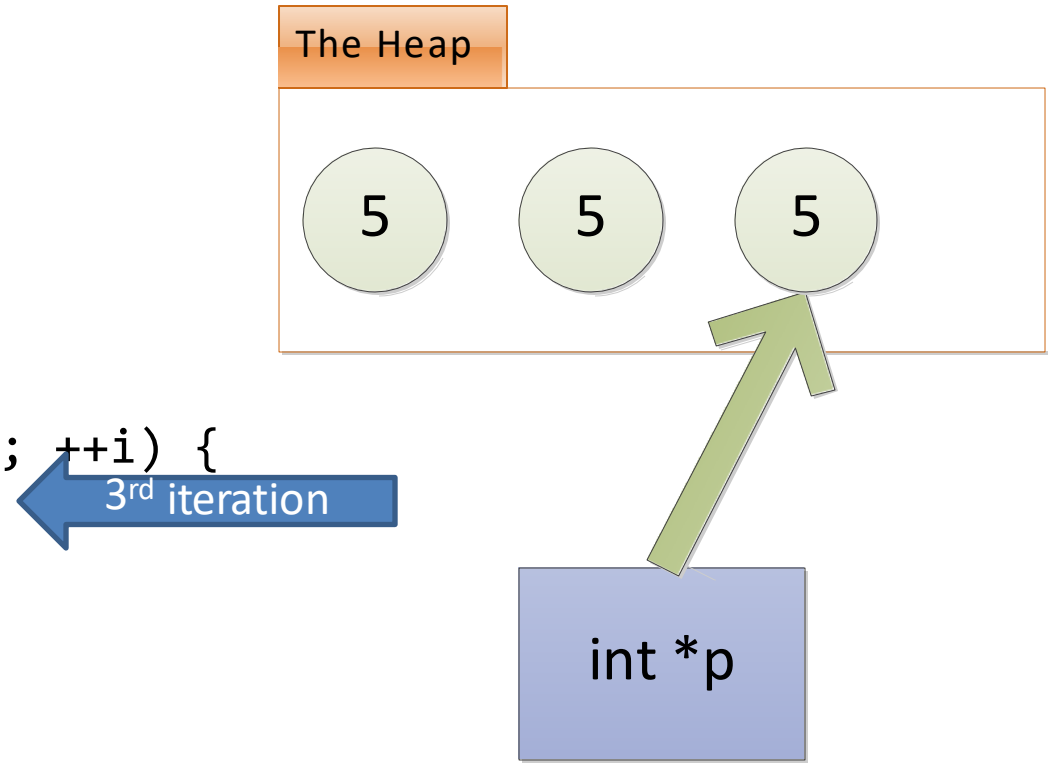
int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
  }
}
```



- If you don’t use de-allocate memory using **delete**, your application will waste memory
- When your program allocates memory but is unable to de-allocate it, this is a **memory leak**

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

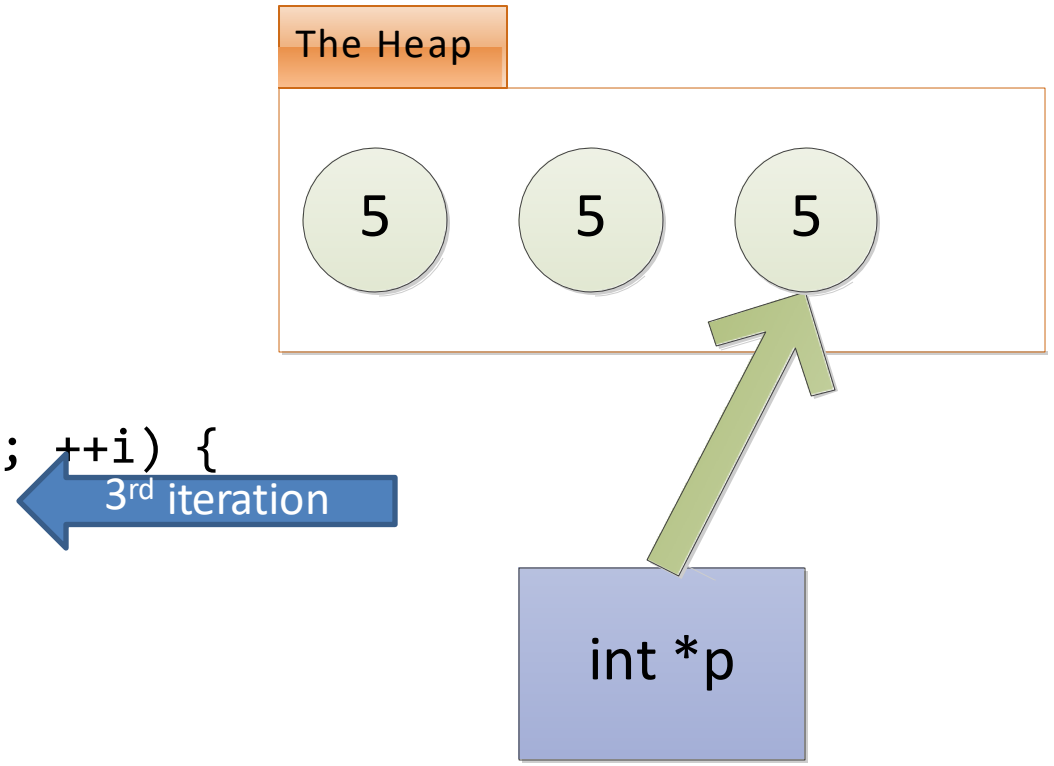
int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
  }
}
```



- Does adding a delete after the loop fix this memory leak?

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

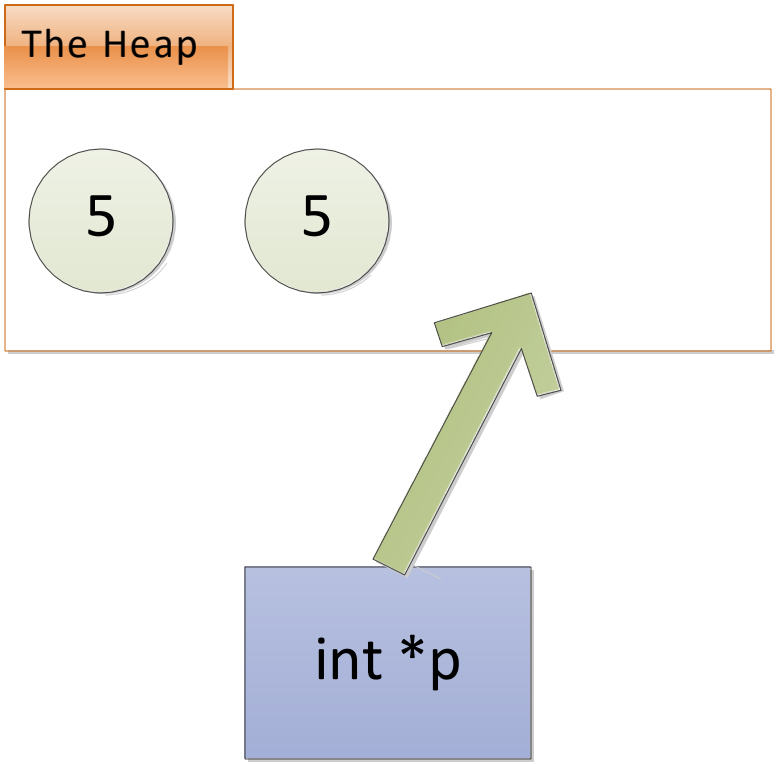
int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
  }
  delete p;
}
```



- Does adding a delete after the loop fix this memory leak?
  - No; only the memory that was allocated on the last iteration gets de-allocated

```
int* getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
  }
  delete p;
}
```



- To fix the memory leak, de-allocate memory within the loop

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
    delete p;
  }
}
```

- To fix the memory leak, de-allocate memory within the loop

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
    delete p;
  }
}
```

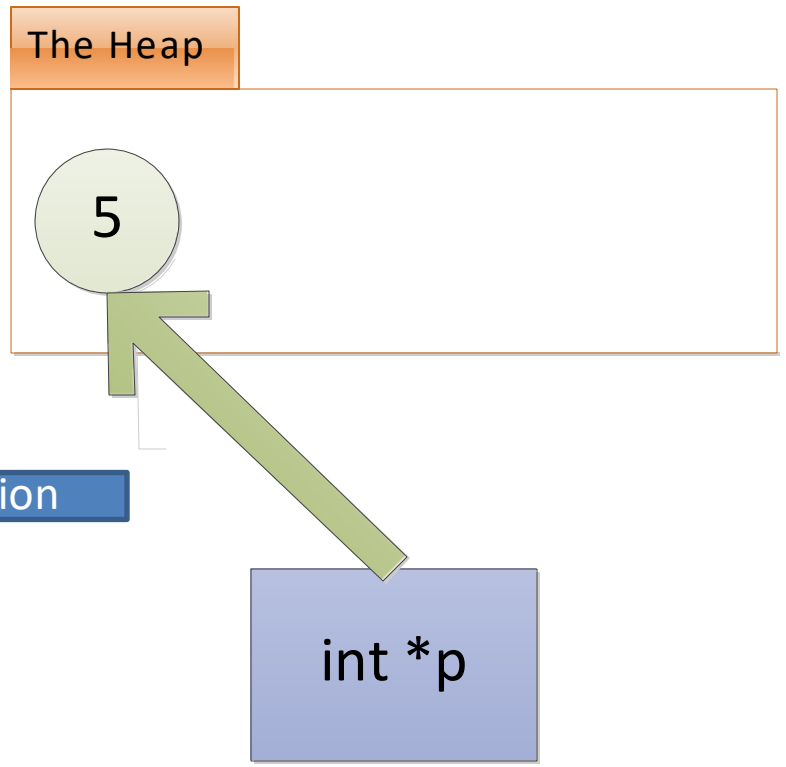


int *p

- To fix the memory leak, de-allocate memory within the loop

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
    delete p;
  }
}
```

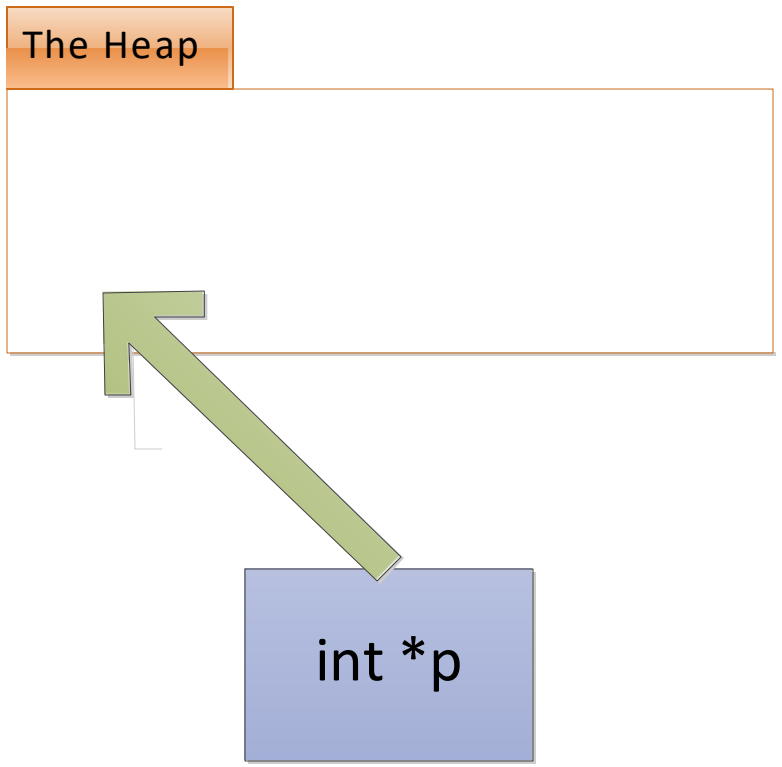


- To fix the memory leak, de-allocate memory within the loop

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
    delete p;
  }
}
```

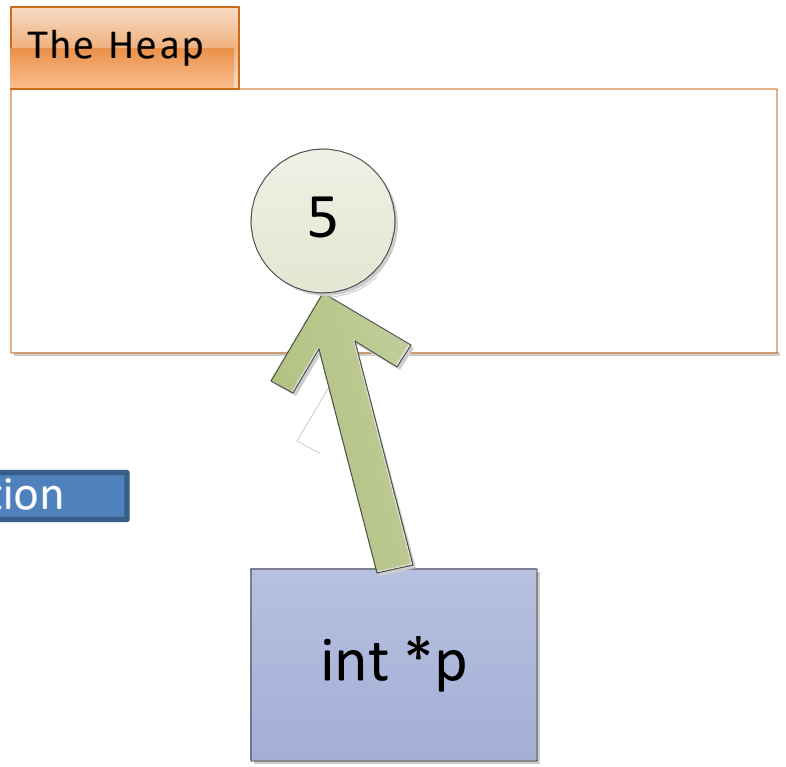
1st iteration



- To fix the memory leak, de-allocate memory within the loop

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
    delete p;
  }
}
```

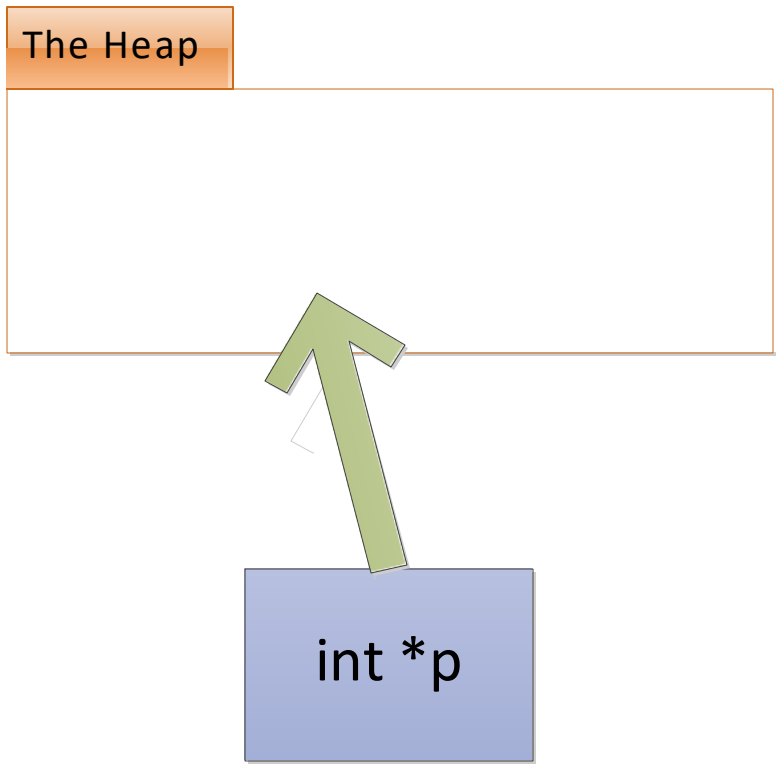


- To fix the memory leak, de-allocate memory within the loop

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
    delete p;
  }
}
```

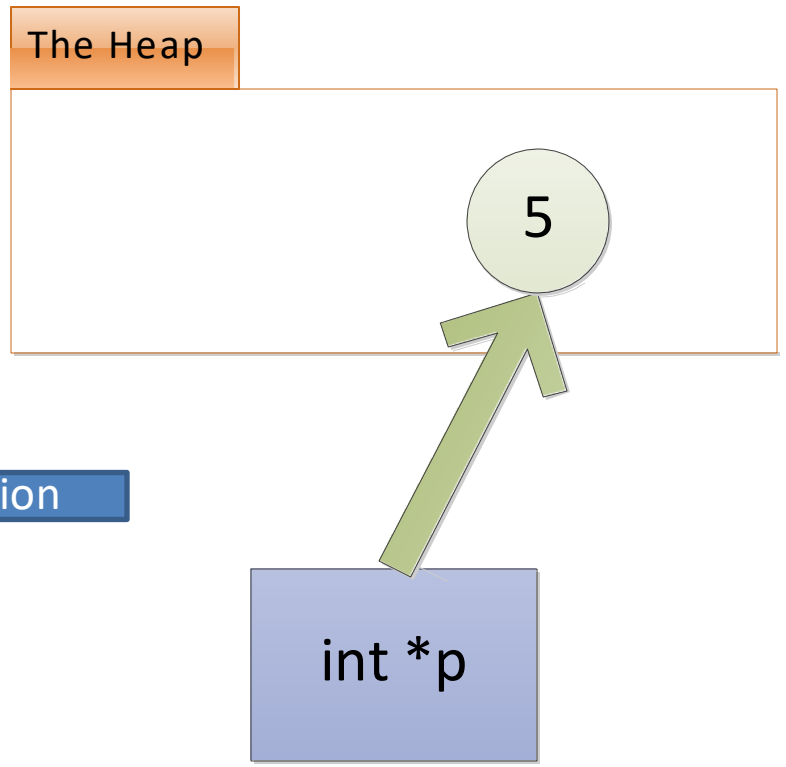
2nd iteration



- To fix the memory leak, de-allocate memory within the loop

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
    delete p;
  }
}
```

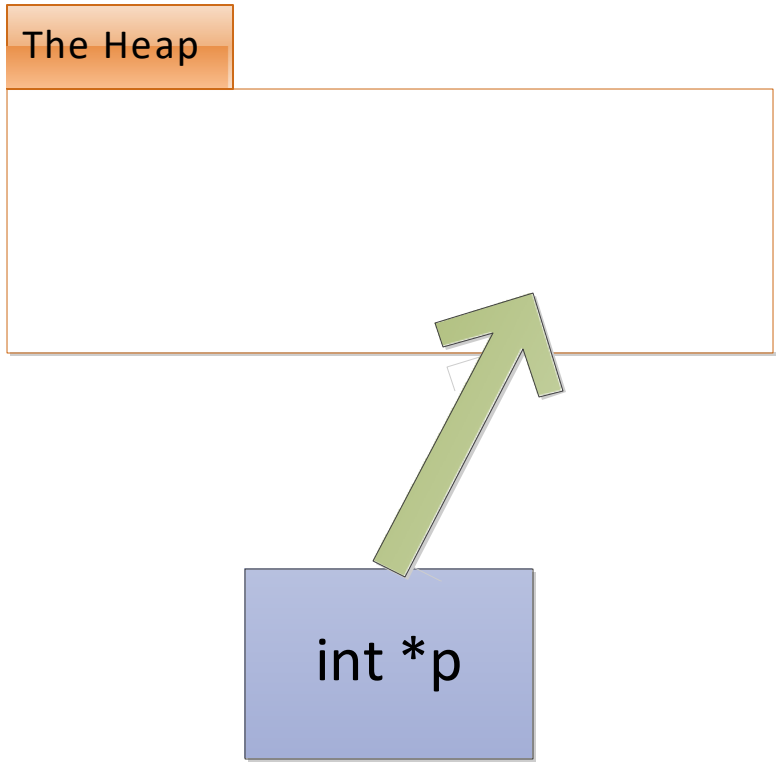


- To fix the memory leak, de-allocate memory within the loop

```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
    delete p;
  }
}
```

3rd iteration

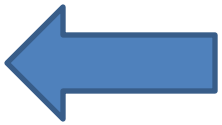


Don’t Use Memory After Deletion




```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *x = getPtrToFive();
  delete x;
  cout << *x << endl; // ???
}
```



Don’t Use Memory After Deletion




```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *x = getPtrToFive();
  delete x;
  cout << *x << endl; // ???
}
```




```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

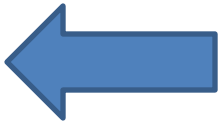
int main() {
  int *x = getPtrToFive();
  cout << *x << endl; // 5
  delete x;
}
```

Don’t delete memory twice




```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *x = getPtrToFive();
  cout << *x << endl; // 5
  delete x;
  delete x;
}
```

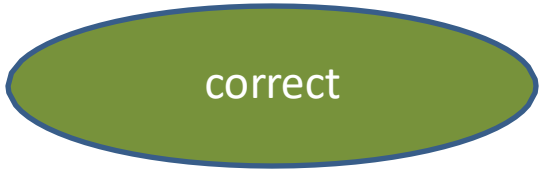


Don’t delete memory twice




```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *x = getPtrToFive();
  cout << *x << endl; // 5
  delete x;
  delete x;
}
```




```
int *getPtrToFive() {
  int *x = new int;
  *x = 5;
  return x;
}

int main() {
  int *x = getPtrToFive();
  cout << *x << endl; // 5
  delete x;
}
```

Only **delete** if memory was allocated by **new**




```
int main() {
   int x = 5;
   int *xPtr = &x;
   cout << *xPtr << endl;
   delete xPtr;
}
```

Only **delete** if memory was allocated by **new**






```
int main() {
   int x = 5;
   int *xPtr = &x;
   cout << *xPtr << endl;
   delete xPtr;
}
```

```
int main() {
   int x = 5;
   int *xPtr = &x;
   cout << *xPtr << endl;
}
```

Allocating Arrays

- When allocating arrays on the stack (using "int arr[SIZE]"), size must be a constant

```
int numItems;
cout << "how many items?";
cin >> numItems;
int arr[numItems]; // not allowed
```

Allocating Arrays

- If we use **new[]** to allocate arrays, they can have variable size

```
int numItems;

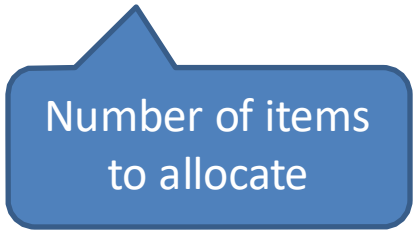
cout << "how many items?";
cin >> numItems;
int *arr = new int[numItems];
```

Type of items in array

Allocating Arrays

- If we use **new[]** to allocate arrays, they can have variable size

```
int numItems;
cout << "how many items?";
cin >> numItems;
int *arr = new int[numItems];
```



Allocating Arrays

- If we use **new[]** to allocate arrays, they can have variable size
- De-allocate arrays with **delete[]**

```
int numItems;
cout << "how many items?";
cin >> numItems;
int *arr = new int[numItems];
delete[] arr;
```

Exercise

Write code to first prompt the user “how many integers” they wish to read in.
The program should allocate enough memory for the array
Read in each integer into the array.
Write out the array to the screen.

Ex: Storing values input by the user

```
int main() {
  int numItems;
  cout << "how many items? ";
  cin >> numItems;
  int *arr = new int[numItems];
  for (int i = 0; i < numItems; ++i) {
    cout << "enter item " << i << ": ";
    cin >> arr[i];
  }
  for (int i = 0; i < numItems; ++i) {
    cout << arr[i] << endl;
  }
  delete[] arr;
}
```

how many items? **3**
enter item 0: **7**
enter item 1: **4**
enter item 2: **9**
7
4
9

2D Arrays

```cpp
#include <iostream>

int main() {
    int rows = 3;
    int cols = 4;

    // Allocate a 2D array
    int** arr2D = new int*[rows];
    for (int row = 0; row < rows; row++) {
        arr2D[row] = new int[cols];
    }

    // Initialize with values
    int counter = 1;
    for (int row = 0; row < rows; row++) {
        for (int col = 0; col < cols; col++) {
            arr2D[row][col] = counter++;
        }
    }

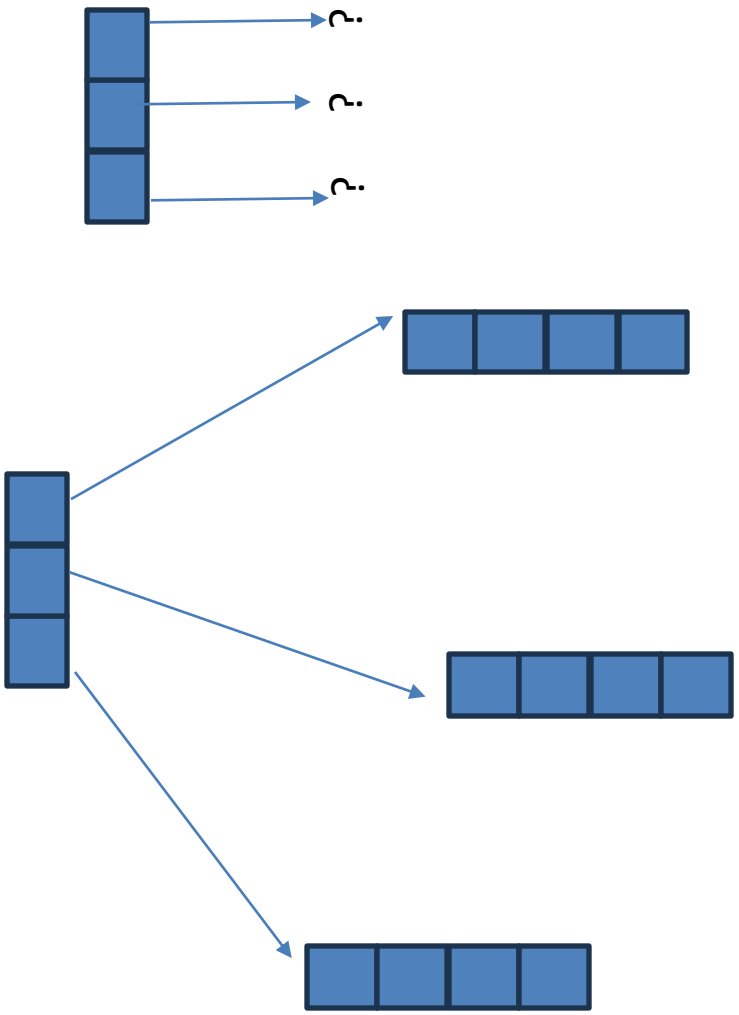
    // Print the array
    std::cout << "2D Array:\n";
    for (int row = 0; row < rows; row++) {
        for (int col = 0; col < cols; col++) {
            std::cout << arr2D[row][col] << " ";
        }
        std::cout << std::endl;
    }

    // Deallocate memory
    for (int row = 0; row < rows; row++) {
        delete[] arr2D[row];
    }
    delete[] arr2D;

    return 0;
}
```

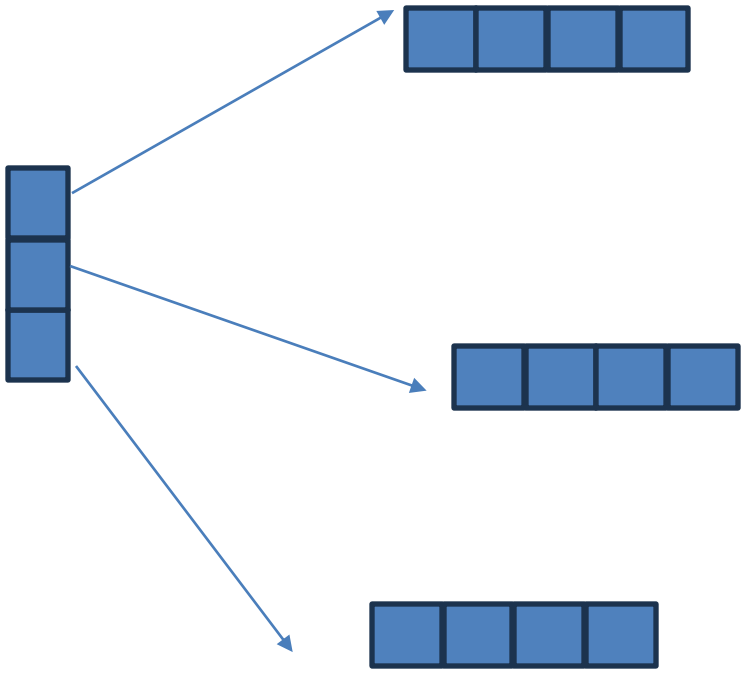
2D Arrays

```
int** arr2D = new int*[rows];
```



```
for (int row = 0; row < rows;
row++) {
    arr2D[row] = new int[cols];
}
```

2D Arrays



```
// Deallocate memory
 for (int row = 0; row < rows; row++)
{
      delete[] arr2D[row];

}
delete[] arr2D;
```

C Memory Management

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Allocate memory for a single integer
    int *ptr = (int*)malloc(sizeof(int));

    if (ptr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    // Use the allocated memory
    *ptr = 42;
    printf("Value: %d\n", *ptr);

    // Free the memory when done
    free(ptr);

    return 0;
}
```

C Memory Management

```c
//Allocate Dynamic Array
#include <stdio.h>
#include <stdlib.h>

int main() {
    int size = 5;

    // Allocate memory for an integer array
    int *arr = (int*)malloc(size * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    // Initialize and use the array
    for (int i = 0; i < size; i++) {
        arr[i] = i * 10;
    }

    // Print the array
    printf("Array elements: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Free the memory
    free(arr);

    return 0;
}
```

C Memory Management

Key Points to Remember

- **Always check if** `malloc()` **returns NULL** - allocation can fail
- **Always call** `free()` for every successful `malloc()` to avoid memory leaks
- **Use** `sizeof()` to get the correct size of the data type
- **Don't use memory after freeing it** - this causes undefined behavior
- **Set pointer to NULL after freeing** (good practice):

- Global Variables

```
int glVar = 10;

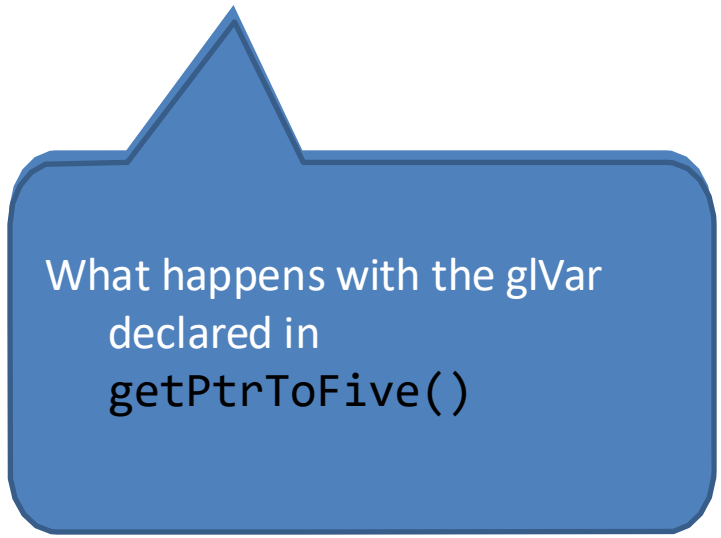

 int main() {
   int *p;
   for (int i = 0; i < 3; ++i) {
     p = getPtrToFive();
     cout << *p << endl;
   }
   glVar=33;
 }
  int *getPtrToFive() {
    int *x = new int;
    *x = 5;
    glVar=20;
    return x;
  }
```



- Global Variables

```
int glVar = 10;


int main() {
  int *p;
  for (int i = 0; i < 3; ++i) {
    p = getPtrToFive();
    cout << *p << endl;
  }
  glVar=33;
}
int *getPtrToFive() {
  int glVar;
  int *x = new int;
  *x = 5;
  glVar=20;
  return x;
}
```



- Global Variables

```
int glVar = 10;


 int main() {
   int *p;
   for (int i = 0; i < 3; ++i) {
     p = getPtrToFive();
     cout << *p << endl;
   }
   glVar=33;
 }
  int *getPtrToFive() {
    int glVar;
    int *x = new int;
    *x = 5;
    glVar=20;
    return x;
  }
```

The compiler creates a new local glVar and only changes that. The global glVar is unchanged

Function Pointers

We can obtain the address of a C function, store it in a function pointer, and then call the function indirectly through that pointer.
However, the manipulation of function pointers is limited to assignment and indirection; we cannot do arithmetic on function pointers.

Function Pointers

```
int Add(int num1, int num2)
{
        return num1 + num2;
}
int main()
{
        //must have the same return type and parameter types as function
        int (*functionPtr)(int , int );

        functionPtr = Add;

        int result = functionPtr(20, 10);
}
```